Excel Shrink: High-Performance Bytestream Parsing and Cleaning of Bloated Excel Spreadsheets

1st Alexander Aue-Johr
Institute of Rural Studies
Thünen-Institute
Braunschweig, Germany
0009-0001-8683-3630

***Abstract*—This paper introduces Excel Shrink, a tool designed to mitigate the pervasive problem of file bloat in Microsoft Excel spreadsheets. By analyzing Excel's Office Open XML structure, we identify common sources of file bloat—such as cells, rows and columns that are empty, contain only whitespace, or have a style assigned that is invisible to the user. We propose a novel, streaming-based, regex-driven approach to reduce file size without altering the visual appearance. Our method employs a state-machine to manage the hierarchy of XML documents without relying on conventional XML parsing techniques by processing XML data incrementally at the bytestream level, and it leverages multithreading for performance improvements. This enables Excel Shrink to outperform Microsoft Excel when loading bloated files, and cleaning plus opening the shrunk file is faster than opening the bloated file directly. Evaluation on a dataset from the German Federal Statistical Office demonstrates that Excel Shrink can achieve file size reductions of up to 99.99% and dramatically improve file loading times. This work not only provides practical insights for developers and end users but also opens new avenues for the efficient management of large-scale semi-structured data in spreadsheet environments.**

***Index Terms*—component, formatting, style, styling, insert.**

I. Introduction

Spreadsheets such as Microsoft Excel are ubiquitous tools for ad-hoc data management and analysis across many domains—from finance to science—even among non-programmers [1]. It is estimated that over one-tenth of the world's population uses spreadsheet software [2]. However, the growing volumes of data have pushed these tools to their limits. Although modern Excel supports up to approximately one million rows per worksheet [2], [3]—a capacity unimaginable in earlier versions—simply increasing this limit does not ensure scalable performance. In practice, spreadsheets often become unresponsive when handling only a few hundred thousand rows [1], [4], underscoring that while spreadsheets remain indispensable, they are inherently ill-suited for managing very large datasets compared to database systems. This tension between ease-of-use and scalability is the motivation for our research.

A particularly frustrating symptom of this limited scalability is *Excel file bloat*—the tendency of `.xlsx` files to become excessively large and slow. Users frequently encounter workbooks that swell in size far out of proportion to any actual data contained within them [5]. Such bloat is not just a storage inconvenience; it also directly degrades performance, causing slow load/save times, heavy memory consumption, and delays in downstream data processing pipelines [6]. Understanding the causes of this bloat is therefore crucial. Prior reports and anecdotal evidence highlight several culprits. One common cause is excess formatting: Excel is notorious for retaining formatting in cells far beyond the used data range, silently inflating file size [7]. In effect, millions of "blank" cells may carry font, color, or border settings that contribute to the XML file, dramatically increasing the package size. Another cause is an overabundance of cell styles. Every unique combination of formatting (font, fill, border, etc.) is stored in a centralized style table; copying worksheets between workbooks or indiscriminate cell styling can accumulate tens of thousands of distinct styles. Excel allows up to 64,000 unique cell styles, and hitting this limit or even a fraction of it can bloat the styles.xml part of the file and trigger errors. [8]. In one documented case, Excel began warning the user that "this workbook has many duplicate styles which can slow performance," prompting a cleanup of hundreds of extraneous style entries [9]. These examples underscore that Excel's own features for flexibility (rich formatting, open XML storage) can turn into liabilities at scale.

The significance of the Excel bloat problem extends beyond file size annoyance—it poses broader scientific and technical challenges. First, it exemplifies the difficulties of managing large semi-structured data in a format not optimized for size or speed. The Office Open XML format (used by .xlsx) trades efficiency for interoperability, using verbose XML to represent spreadsheet content. When a workbook contains huge numbers of cells or lengthy text, the XML overhead (tags, attributes, and repetition) adds substantial bulk. A DOM parser, which builds an in-memory tree of the entire XML, can easily exhaust memory on multi-hundred-thousand-row sheets. For example, the memory consumption of a XML DOM parser (like `readxl`) can exceed ten times the size of the uncompressed worksheet (30 GB for 6.000.000 rows) [10]. Streaming SAX parser avoids high memory usage but suffer from high CPU overhead due to handling a flood of callback events [10].

Such scenarios reflect the mismatch between Excel's data model and big-data processing needs. Processing such bloated

XML files is computationally expensive: parsing XML requires reading and interpreting deeply nested structures, which can strain I/O and memory. Indeed, standard XML parsing in memory (DOM) would load the entire spreadsheet into a tree, potentially consuming gigabytes of RAM for a bloated file. This is aggravated by Excel-specific XML components like the Shared Strings table (which holds all text entries) and the Styles table – both can grow very large and must be parsed or loaded before data can be accessed. As a result, reading a large .xlsx file with conventional libraries often becomes a bottleneck [11]. Second, the issue brings to light fundamental challenges in language theory and parsing. Excel's file format—essentially a collection of XML documents—is a context-free language (Type-2 in the Chomsky hierarchy). Parsing such documents correctly requires managing recursive tag structures and dependencies (e.g., matching `<row>` and `</row>` tags or resolving references in shared string tables). It is mathematically impossible to accurately parse or validate XML using only regular expressions or simplistic line-by-line methods, since regular languages (Type-3) cannot handle arbitrary nested structures [12], [13]. Consequently, robust tools for processing `.xlsx` files at scale must employ proper XML parsing strategies rather than relying on ad-hoc techniques.

This paper situates the Excel bloat problem within the context of database research and proposes technical approaches to mitigate it. We begin by analyzing the internal structure of Excel's Open XML format to identify the causes of file bloat and review related work and online resources regarding bloat in Excel files. We employ manual guides as well as tools such as Microsoft Excel's Performance Tab and the Excel Inquire Add-In, and also trial versions of proprietary utilities like NXPowerLite Desktop, to assess their file size reduction capabilities.

After applying these manual guides and automatic tools to real-world Excel files, we perform a qualitative analysis of the XML code to examine the changes responsible for file size reduction. By comparing the visual integrity of the files before and after cleaning—and incrementally reverting modifications to pinpoint when visual errors disappeared—we gain insight into Excel's internal file structure and determine which cleanup operations are safe. Building on this reverse-engineering process, we identify a set of safe modifications and implement them in our own protptype tool - Excel Shrink Analyzer - using regular DOM-XML parsing for fast prototyping, to test these changes across a large number of files and validate the outcome by file size reduction and if the Spreadsheet still looks the same, to find the set of modifications that are safe and effective.

Following the prototype phase, we develop a tool applying the same modifications, but this time efficiently, using a state-machine-guided, regex-driven parser designed to clean bloated Excel files. This parser operates directly on the raw XML byte stream to overcome the limitations of conventional XML parsers when dealing with large, deeply nested structures. We evaluate the performance of this tool — referred to as Excel Shrink — on the same Dataset a series of Excel files by comparing the loading times of pre-cleaned files versus the original files when opened with standard libraries and Microsoft Excel itself.

To guide our study, we pose the following research questions:

1) **(RQ1)** Are the root causes of Excel file bloat already well understood and documented in existing research, or is further empirical analysis needed to fully identify and quantify these causes?
2) **(RQ2)** Can a regex-driven, state-machine–guided parser overcome the typical limitations of regular expressions in processing structured XML data (e.g., prevent overmatching and preserve XML validity)?
3) **(RQ3)** Does precleaning bloated Excel files with such a method significantly improve overall loading time (*i.e.*, cleaning + loading) compared to loading the original file directly in Excel?

Through this exploration, we aim to bridge the gap between Excel's convenience and the demands of very large-scale data management. Ultimately, our work aspires to improve the reliability and scalability of processing large Excel files by providing both developers and end users with actionable insights and practical tools. Developers gain a deeper understanding of the causes of file bloat and strategies for efficient cleanup, while Excel users receive a means to pre-clean files before further processing or to integrate Excel Shrink into automated workflows.

II. Background

A. Excel's Open XML Format

Excel workbooks (.xlsx) are stored in the Office Open XML format, which organizes spreadsheet data into multiple XML parts within a ZIP container. Notably, each worksheet is an XML file (sheet1.xml, sheet2.xml, etc.) containing the cell data (values, formulas) for that sheet [14]. In addition, there are global parts such as workbook.xml (overall workbook metadata like the names of the sheet and their order), styles.xml (definitions of cell, row and column formats), and sharedStrings.xml (the table of reoccuring texts in the file). The Shared String Table is especially important for understanding Excel's storage mechanism. It is a construct that contains one instance of each unique text string appearing in the workbook. Instead of writing repeated text values in every cell, Excel stores each distinct string once in sharedStrings.xml and then references it by index from the sheet files. This design aims to reduce file size and memory usage when many cells contain duplicate text (e.g., a column of repeating categorical values) [15], [16]. Each text cell in a sheet XML may look like <c r="A1" t="s"><v>42</v></c>. This means that the cell at position A1 does not contain the number 42 itself. Instead, the attribute t="s" marks the cell's content as a shared string reference — the <v> tag holds the index (here: 42) of the actual text stored in the file sharedStrings.xml. When Excel opens the worksheet, it looks up index 42 in that shared string table to retrieve and display the correct text in the cell. In

scenarios with highly repetitive text, the shared string table yields smaller files.

Another central component is the Styles Part (styles.xml), which defines all cell formatting used in the workbook. Excel supports rich formatting options for cells—such as fonts, colors, number formats, and borders—and manages these through style records. Each style record represents a specific combination of formatting properties and can be shared among multiple cells, rows, or columns. Instead of duplicating formatting information, these elements simply reference the corresponding style record via the s attribute. [17].

B. Prior Observations and Literature

Despite the prevalence of the Excel bloat problem in practice, there is scant academic literature that directly addresses it. To our knowledge, no peer-reviewed study thus far has focused on the internal inefficiencies of Excel's XML format or the mitigation of bloated spreadsheets. This is consistent with the fact that the database research community has traditionally viewed spreadsheets as end-user tools rather than enterprise-scale data management systems. However, indirectly, several lines of work and industry reports touch on related issues. Microsoft's official guidelines and support articles acknowledge problems such as the "Too many different cell formats" error (triggered by style overflow) [8] and provide best practices for reducing workbook size (e.g., removing unused styles, turning off pivot cache, etc.). The Excel user community (blogs, forums) is rife with case studies of bloated files and remedies. For example, Power [7] enumerates common reasons for oversized files and recommends using a ZIP tool to inspect which internal parts are largest - an approach that reveals whether data, formatting, or media are to blame. On Stack Overflow, Superuser and ExcelForum, numerous Q&A threads document instances of mysterious file growth, often resolved by cleaning formats or rebuilding the workbook from scratch [18]–[21] . These sources, while not formal research, provide valuable insight into the root causes of bloat and confirm that our problem formulation is grounded in reality. However the solutions they provide are often manual, some of them utalizing the Excel built in features, recently introduced features like the excel check performance tab or the older Excel Inquire Add In, which are not suitable for automation pipelines and not scalable for large numbers of files. Yet the causes in the XML structure of the Excel file is not addressed in these sources and can only be identified by reverse engeneering the cleanup tools or Microsoft excel itself by try and error removing parts of the XML file and checking if the file still looks the same.

In the academic sphere, there is growing recognition that spreadsheets and databases need not be disjoint worlds. Some research efforts have looked at enhancing spreadsheets to handle larger datasets by leveraging database technology in the background. DataSpread [1] and Hillview [4] are two systems that retain an Excel-like interface but manage data in a relational database to improve scalability.

While these works are not about Excel's file format per se, they underscore a broader point: when spreadsheets approach the size of databases, one must apply database-like solutions. Our work aligns with this philosophy by treating Excel bloat and processing as a data management problem. Instead of dismissing bloated spreadsheets as user error, we view them as an important issue in large-scale semi-structured data processing that warrants efficient techniques.

III. Related Work

A substantial body of research in top-tier venues (VLDB, SIGMOD, ICDE) has explored techniques to *improve performance* on large or complex workbooks. There have also been several efforts to reduce *Excel file size*, with various online resources discussing this topic. We briefly review key findings below, focusing on (i) methods for minimizing file size, (ii) approaches to accelerating computation and user interactions for large spreadsheets, (iii) optimizations for parsing the XML-based `.xlsx` format, and (iv) highlights of Aditya Parameswaran's contributions in bridging spreadsheet and database technologies.

Minimizing Excel File Size and Bloat. Microsoft's transition from the binary `.xls` format to `.xlsx` (Office Open XML) introduced compression and a shared string table to reduce repeated data [22], [23]. Further reduction is achieved in the `.xlsb` (binary) format, which stores data in a compact binary container at the cost of certain compatibility constraints [6], [24].

However, there is a lack of peer-reviewed studies specifically quantifying how "bloat" (whitespace values, invisible formatting, rows and column definitions outside of the visible content) affects Excel file size [25]. Instead, most advice on the subject comes from practitioners sharing their experiences. For example, Microsoft's support article on cleaning excess cell formatting provides practical tips for reducing file size and improving performance [24]. Additionally, discussions on forums such as MrExcel illustrate that users have long noticed dramatic size reductions after clearing unused formatting or trimming the "used range" [26]. Popular Excel blogs from UpSlide [27], The Bricks [28] and Chandoo [29] further detail these methods based on accumulated user experience rather than formal experimentation.

Several proprietary tools have been developed to address Excel bloat automatically. Notable examples include UpSlide's Automation Software, which offers a one-click "Smart Clean" feature [27]; NXPowerLite Desktop [30]; Sobolsoft's Excel File Size Reduce Software [31]; and online services such as WeCompress [32]. Other tools, such as Kernel for Excel Repair [33], have also been mentioned in user communities for their file compression capabilities.

Microsoft has also developed two tools to assist users. One such tool is **Excel Inquire**, an add-in introduced with Excel 2013 that provides a range of auditing features—including Workbook Analysis, Relationship Diagrams, and the Clean Excess Cell Formatting command. O'Beirne notes that the *Clean Excess Cell Formatting* command removes excess formatting

but does not indicate beforehand what will be cleared, nor does it report afterwards what was removed [34].

More recently, Microsoft introduced the **Excel Performance Tab**, a built-in feature designed to monitor workbook performance by identifying inefficiencies and offering cleaning recommendations. However, the Performance Tab has received negative user feedback; several users report that its forced "Check Performance" pop-up and the associated reformatting remove intentional formatting and disrupt workflows [35], [36] and also Excel users report that their pixel art is deleted [35] while the Microsoft Support Site even mentions a warning that pixel art will be removed by the tool [36].

In contrast, our paper provides a comprehensive scientific analysis of the root causes of Excel file bloat in XML-based workbooks. To address this gap, we introduce *Excel Shrink*— a command-line tool designed for automation pipelines. Unlike proprietary tools that are only accessible through a graphical user interface and sometimes remove too many elements (thereby altering the visual appearance) or too few (leaving the file size large), Excel Shrink selectively reduces file size by eliminating only those elements that do not affect the workbook's visual integrity.

Performance in Handling Large Spreadsheets. Rahman et al. [37] conducted a comprehensive study comparing Excel, LibreOffice Calc, and Google Sheets on sizable datasets. They revealed that despite nominal row limits of one million, Excel's GUI would often freeze with mere tens of thousands of rows or extended formula usage. Similar findings appear in other benchmarking efforts [2], highlighting the lack of indexing in typical spreadsheet software. Addressing this, "DataSpread" [1] (VLDB 2015) and "Hillview" [4] (PVLDB 2019) offload data management to database engines or distributed clusters, respectively, while retaining a spreadsheet-like interface. These architectures stretch the notion of a "spreadsheet" to hundreds of millions (or billions) of rows by introducing parallelization, indexing structures, or incremental computation. In a different line of work, Bendre et al. [38] proposed an *anti-freeze* model for formula recalculation, allowing asynchronous updates so the user interface does not lock while large spreadsheets recompute.

Efficient Excel XML Processing. As modern Excel files (`.xlsx`) are essentially XML (SpreadsheetML) within a ZIP container, *parsing* can become a bottleneck for large sheets. Classical DOM-based XML parsing can exhaust memory on multi-hundred-thousand-row spreadsheets, whereas SAX-based streaming, though more memory-friendly, suffers from high CPU overhead through numerous event callbacks [10]. A hybrid parser that combines DOM and SAX parsing, along with regular expressions for extracting the relevant file contents (e.g. `openxlsx` in R), can still be slow and memory-intensive under worst-case scenarios. By contrast, parsers relying exclusively on DOM parsing (e.g. `readxl` in R) may achieve faster runtime performance but often at the cost of higher memory usage. For instance, the memory consumption of `readxl` can exceed ten times the size of the uncompressed worksheet [10]. To address this, Henze et al. [10] introduced *SheetReader*, a specialized streaming parser that integrates decompression and XML scanning, and parallelizes chunk-level parsing. This yields dramatically lower memory footprints (often $40\times$ less) and faster loads ($3\times$ or more) compared to standard libraries. Similar to this Excel Shrink also aims to reduce the memory footprint by streamwise processing of small chunks of the XML file and only keeping the relevant parts in memory.

Comparisons of Parsing Approaches and Tools. These studies collectively show a trade-off between *DOM-based* simplicity (at the cost of high memory usage) and fully *SAX-based* pipelines (often incurring higher CPU overhead). Hybrid or domain-specific solutions—like regex-based extraction or streaming chunk processors that skip unneeded style markup—strike a middle ground [10], [39]. Meanwhile, broad tool evaluations (Microsoft Excel vs. Google Sheets vs. LibreOffice Calc) confirm that no mainstream spreadsheet program handles large data sets efficiently out of the box [37].

Parameswaran's Contributions: Bridging Spreadsheets and Databases. Aditya Parameswaran and collaborators have researched integrating spreadsheets with database backends to achieve both usability and scalability [1], [40]. *DataSpread* preserves the familiar Excel-like interface while relying on a relational DBMS for storage and large-scale computations, enabling interactive performance even at billions of cells. This approach can be seen as bypassing Excel's internal storage and recalculation limits by adopting database features (indexing, query optimization, concurrency control). In related work on "anti-freeze" recalculation, Parameswaran's group explored asynchronous formula evaluation with incremental updates, preventing interface lock-ups for large or complex dependency graphs [38]. Additionally, they highlight practical tactics like saving spreadsheets as binary (`.xlsb`) instead of `.xlsx` to mitigate bloat, limiting volatile formulas (e.g. `OFFSET`), and offloading heavy operations to databases where possible [6]. All these strategies offer evidence that a combination of advanced parsing, background recalculation, and robust data management techniques can significantly enhance performance for real-world Excel workloads.

Summary From straightforward file-format optimizations (`.xlsb` compression) to *architectural* overhauls (DataSpread, Hillview) and specialized parsers (SheetReader), the research consensus is that Excel's native engine and file format alone do not scale gracefully. Modern solutions increasingly combine streaming-based XML handling, database or distributed architectures, and asynchronous computation to push spreadsheet interactivity far beyond Excel's historical limits. These insights strongly motivate the need for further improvements in `.xlsx` bloat reduction and high-performance parsing directly addressed by our proposed approach in this paper.

IV. Methodology

Excel-Shrink is a stream-oriented XLSX cleaner that removes visually inconsequential XML while preserving user-visible layout semantics (e.g., fill/border styles that convey meaning), and merges superfluous column definitions. The

system operates directly on the bytestream of the zipped OOXML container: it parses `xl/workbook.xml` and per-sheet `xl/worksheets/sheet*.xml` streams with regular expressions instead of loading full XML trees into memory, applies workbook-specific rewriting rules derived from `xl/styles.xml` and `xl/sharedStrings.xml`, and re-emits a compacted archive.

We first formalise inputs and artefacts, then describe pipeline stages.

A. Inputs, Outputs, and Notation

Inputs. *Path* may be a single XLSX file, a directory, or an HTTPS/HTTP URL. Optional flags control recursion, multiprocessing, chunk size for streaming, sheet filters, and whether only the workbook is cleaned.

Outputs. A cleaned XLSX written either to a file path or into an output directory, preserving all unmodified parts of the original OOXML.

Core artefacts.

- *Workbook XML* (`xl/workbook.xml`): contains `<definedNames>` nodes.
- *Sheet XML* (`xl/worksheets/sheet*.xml`): contains `<cols>`, `<sheetData>`, `<mergeCells>`.
- *Styles* (`xl/styles.xml`): provides `<cellXfs>` with fill/border/apply flags.
- *Shared Strings* (`xl/sharedStrings.xml`): enables identification of pure-whitespace string table entries.

Workbook-specific patterns. From `styles.xml` and `sharedStrings.xml` we derive:

1) sets of style indices with *fill* or *border* or with neither but *font-only*;
2) regex-like predicates for: (i) whitespace cells, (ii) consecutive empty cells/rows, (iii) attributes irrelevant for empty rows (*spans*, vendor *dyDescent*, etc.), and (iv) presence of *hidden* rows/columns.

B. Deriving Workbook-Specific Patterns

Our worksheet cleaning approach adopts a two-stage cleaning procedure based on a chunking strategy. In the initial preprocessing phase, individual chunks are analysed and cleaned using lightweight operations that do not require any global context from the worksheet. Some of these operations require knowledge of the workbook structure, others can be applied independently to each worksheet. Despite their simplicity, these early operations are effective in eliminating a large portion of file bloat from Excel spreadsheets.

Such operations involve the removal of visually inconsequential cells and rows— cells that contain no content and no layout-relevant styling, such as background fills or borders. Font-only styling (e.g., font family or size) or filled with whitespace characters is insufficient to classify a cell as visually relevant. By filtering out these cells during chunk-level processing, we reduce both the computational burden and the memory footprint of subsequent operations.

To identify and remove these visually inconsequential cells, we need to know which styles correspond to visually empty formatting and which shared strings correspond to whitespace-only content.

After extracting the style definitions from styles.xml and the shared strings from sharedStrings.xml during the initial context extraction phase, we can compile regex patterns that match cells and rows that are visually empty. The compilation of these patterns is summarised in Algorithm 1.

After extracting the style definitions from `styles.xml` and the shared strings from `sharedStrings.xml` during the initial context extraction phase, we construct several sets of indices that determine which cells or rows can safely be removed. These sets and the resulting regular expressions are compiled in Algorithm 1.

Let S_{fb} denote the set of all *fill/border* style indices, i.e. those `xf` records where either the `fillId` or `borderId` attribute is non-zero. For example, if the parsed style table (`styles.xml`) contains

$$XFS[0].\text{fillId} = 0, XFS[1].\text{fillId} = 3, XFS[2].\text{borderId} = 2$$

then $S_{\text{fb}} = \{1, 2\}$. These indices are joined into a single alternation string $\text{FB} =$ `"1|2"`, which is used to form the regular expression

$$p_{\text{fbh}} = \texttt{"(?:s="(?:1|2)"|"hidden="1")"}$$

matching a row attribute that either has style with a set fill or border or a row attribute that is hidden used to find the last visually relevant row.

Analogously, S_{font} represents the set of *font-only* styles—those that apply no fill or border at all. For instance, if

$$XFS[3].\text{fillId} = 0, \quad XFS[3].\text{borderId} = 0,$$
$$XFS[4].\text{fillId} = 0, \quad XFS[4].\text{borderId} = 0,$$

then $S_{\text{font}} = \{3, 4\}$ is incorporated into the patterns

$$p_{\text{ra}} = \texttt{\textbackslash s *(spans="[^"]+")}$$
$$\texttt{|[^:]+:dyDescent="[^"]*"}$$
$$\texttt{|s="(?:3|4)")}$$

$$p_{\text{e}} = \texttt{(?:<c\textbackslash b (?:(?![^>]*s=)[^>]*)}$$
$$\texttt{|[^>]*s="(?:3|4)"}$$
$$\texttt{[^>]*)/>)+}$$

which respectively match redundant row attributes and consecutive font-only empty cells.

Next, W is the set of *shared string indices* that correspond to whitespace-only values (e.g. cells containing a single space or tab). For example, if `sharedStrings.xml` contains

```
<si><t> </t></si>
<si><t>  </t></si>
<si><t>hello</t></si>
```

then $W = \{0, 1\}$ and $\text{WS} =$ `"0|1"`.

These indices produce the regexes

$$p_{\text{ws}} = \texttt{<v>(?:0|1)</v>}$$

$$p_{\text{wc}} = \texttt{(<c\textbackslash b [^>]*t="s"[^>]*?)><v>(?:0|1)</v></c>}$$

which match the corresponding whitespace value tags and their enclosing <c> elements.

All resulting expressions p_{ra}, p_{e}, p_{fbh}, p_{ws}, and p_{wc} are compiled into the regex set R. Together with the mapping XFS from style IDs to `xf` records, they form a complete *pattern context* that drives the subsequent byte-stream cleaning phase. This allows the cleaner to efficiently detect and remove visually empty cells and rows purely through regular-expression matching—without ever parsing the XML structure.

Algorithm 1: Consolidated construction: compute alternations and build dependent regexes in the same branch.

Function BUILDWORKBOOKPATTERNS*($B_{stylesXml}$, $B_{sharedStringsXml}$)*:
- $xfs \leftarrow$ parseCellXfs($B_{\text{stylesXml}}$)
- $S_{\text{fb}} \leftarrow \{\, i \mid xfs[i].\text{fillId} \neq 0 \vee xfs[i].\text{borderId} \neq 0 \,\}$
- $S_{\text{font}} \leftarrow \{\, i \mid (xfs[i].\text{fillId} \in \{0, \texttt{None}\}) \wedge (xfs[i].\text{borderId} \in \{0, \texttt{None}\}) \wedge \neg xfs[i].\text{applyFill} \wedge \neg xfs[i].\text{applyBorder} \,\}$
- $W \leftarrow$ INDICESOFWHITESPACESI($B_{\text{sharedStringsXml}}$)
- XFS $\leftarrow \{\text{"0"} \mapsto xfs[0],\ \text{"1"} \mapsto xfs[1], \ldots\}$
- **if** $|S_{\text{font}}| > 0$ **then**
  - OFS $\leftarrow$ `join(`S_{font}`, |)`
  - $p_{\text{ra}} \leftarrow$ `'\s *(\bspans ="[^"]+?"|' + '\b [^\s :]*?:dyDescent="[^"]*"|' + '\bs ="(?:' + OFS + ')")'`
  - $p_{\text{e}} \leftarrow$ `'(?:<c\b (?:(?![^>]*\bs =)[^>]*|[^>]*' + '\bs ="(?:' + OFS + ')"[^>]*)/>)+'`
- **else**
  - $p_{\text{ra}} \leftarrow$ `'\s *(\bspans ="[^"]+?"|' + '\b [^\s :]*?:dyDescent="[^"]*")'`
  - $p_{\text{e}} \leftarrow$ `'(?:<c\b (?![^>]*\bs =)[^>]*/>)+'`
- **if** $|S_{\text{fb}}| > 0$ **then**
  - FB $\leftarrow$ `join(`S_{fb}`, |)`
  - $p_{\text{fbh}} \leftarrow$ `'(?:s="(?:' + FB + ')"|"hidden="1")'`
- **else**
  - $p_{\text{fbh}} \leftarrow$ `'"hidden="1"'`
- **if** $|W| > 0$ **then**
  - WS $\leftarrow$ `join(`W`, |)`
  - $p_{\text{ws}} \leftarrow$ `'<v>(?:' + WS + ')</v>'`
  - $p_{\text{wc}} \leftarrow$ `'(<c\b [^>]*?\bt ="s"[^>]*?)><v>(?:' + WS + ')</v></c>'`
- **else**
  - $p_{\text{ws}} \leftarrow$ None
  - $p_{\text{wc}} \leftarrow$ None
- $\mathcal{P} \leftarrow \{p_{\text{ra}}, p_{\text{e}}, p_{\text{fbh}}, p_{\text{ws}}, p_{\text{wc}}\}$ with COMPILE for all non-`None`
- **return** $\langle \mathcal{P}$, XFS$\rangle$

These patterns parameterise all sheet and workbook transformations that follow, ensuring we preserve semantically relevant visual styling.

C. Streaming Sheet Pre-processing

We implement our spreadsheet parser streaming-based, processing the XML data incrementally to minimize memory consumption and permit early discarding of unnecessary file sections. Below, we focus on how the parser transitions between states (i.e., how each state function decides to proceed), deferring lower-level details of in-state processing until later sections (Algorithm 2).

Algorithm 2: Streaming extraction of sheet components

Function
PREPROCESSSHEETSTREAMWISE*($sheetStream$, $\mathcal{P}$,* chunkSize*)*:
- $state \leftarrow$ IN_WORKSHEET; $b \leftarrow \epsilon$;
- $B_{\text{preCols}}, B_{\text{cols}}, B_{\text{preSD}}, B_{\text{sheetData}}, B_{\text{preMC}}, B_{\text{mergeCells}}, B_{\text{tail}} \leftarrow \epsilon$;
- $\mathcal{T} \leftarrow \varnothing$; *processed tags*
- $\mathcal{T} \leftarrow \varnothing$; *processed tags*
- $\mathcal{D}_{col} \leftarrow$ newRangeMap(); *column defs (map)*
- $m_{\max} \leftarrow \bot$;
- $\mathcal{O} \leftarrow$ {`<cols>`, `<sheetData>`, `<mergeCells>`, `</worksheet>`};
- **while** $state \neq$ FOUND_END **do**
  - $b \leftarrow b \,\|\, \text{read}(sheetStream,$ chunkSize$)$;
  - **if** $state =$ IN_WORKSHEET **then**
    - $R \leftarrow \mathcal{O} \setminus \mathcal{T}$;
    - $\langle tag,\ i_{\text{tag}},\ j_{\text{tag}} \rangle \leftarrow \text{nextTag}(b, R)$;
    - **if** $tag \neq \bot$ **then**
      - **if** $tag =$ `<cols>` **then** $B_{\text{preCols}} \leftarrow B_{\text{preCols}} \,\|\, b[0{:}i_{\text{tag}})$
      - **if** $tag =$ `<sheetData>` **then** $B_{\text{preSD}} \leftarrow B_{\text{preSD}} \,\|\, b[0{:}i_{\text{tag}})$
      - **if** $tag =$ `<mergeCells>` **then** $B_{\text{preMC}} \leftarrow B_{\text{preMC}} \,\|\, b[0{:}i_{\text{tag}})$
      - $b \leftarrow b[j_{\text{tag}}{:}|b|)$;
      - $state \leftarrow \text{nextState}(tag)$;
      - $\mathcal{T} \leftarrow \mathcal{T} \cup \{tag\}$;
  - **else if** $state =$ IN_COLS **then**
    - $\langle c,\ b,\ state,\ \mathcal{D}_{col} \rangle \leftarrow$ CONSUMECOLS$(b,\ \mathcal{P}, \mathcal{D}_{col})$
    - $B_{\text{cols}} \leftarrow B_{\text{cols}} \,\|\, c$
  - **else if** $state =$ IN_SHEETDATA **then**
    - $\langle c,\ b,\ state \rangle \leftarrow$ CONSUMESHEETDATA$(b, \mathcal{P})$
    - $B_{\text{sheetData}} \leftarrow B_{\text{sheetData}} \,\|\, c$
  - **else if** $state =$ IN_MERGECELLS **then**
    - $\langle c,\ b,\ state,\ m_{\max} \rangle \leftarrow$ CONSUMEMERGECELLS$(b,\ m_{\max})$
    - $B_{\text{mergeCells}} \leftarrow B_{\text{mergeCells}} \,\|\, c$
  - **else**
    - $B_{\text{tail}} \leftarrow B_{\text{tail}} \,\|\, b[0{:}|b|)$;
    - $b \leftarrow \epsilon$;
    - **break**
- $\mathcal{M} \leftarrow \{\mathcal{D}_{col}, m_{\max}\}$
- **return** Preprocessed$\begin{pmatrix} B_{\text{preCols}},\ B_{\text{cols}},\ B_{\text{preSD}},\ B_{\text{sheetData}}, \\ B_{\text{preMC}},\ B_{\text{mergeCells}},\ B_{\text{tail}}, \mathcal{M} \end{pmatrix}$;

Function CONSUMESHEETDATA*($b, \mathcal{P}$)*:
- $i \leftarrow \text{find}\big(b,$ `</sheetData>`$\big)$
- **if** $i \geq 0$ **then return** $\langle$PREPROCESSSD$(b[0{:}i], \mathcal{P}),\ b[i{:}],$ IN_WORKSHEET$\rangle$
- $k \leftarrow \text{rfind}\big(b,$ `<row>`$\big)$
- **if** $k \geq 0$ **then return** $\langle$PREPROCESSSD$(b[0{:}k], \mathcal{P}),\ b[k{:}],$ IN_SHEETDATA$\rangle$
- **return** $\langle \epsilon,\ b,$ IN_SHEETDATA$\rangle$

a) States Overview: We define six main states in our parser:

- `BEFORE_COLS`: The parser has not yet encountered the `<cols>` element. It searches for either `<cols>` or `<sheetData>`.
- `IN_COLS`: The parser found `<cols>` and is processing column definitions.
- `BEFORE_SHEET_DATA`: The parser has finished reading (or did not detect) the `<cols>` node and is searching for the first `<sheetData>` element.
- `IN_SHEET_DATA`: The parser is processing the main

worksheet data (rows and cells).

- AFTER_SHEET_DATA: The parser has detected the closing </sheetData> and is collecting any subsequent chunks until the end of the file.
- END_OF_FILE: The parser has reached the end of the stream, and no further parsing is needed.

b) In between nodes: In BEFORE_COLS, BEFORE_SHEET_DATA, or AFTER_SHEET_DATA, the parser looks for tag boundaries (e.g., <cols>, <sheetData>) or the end of file. When it detects such a tag at position i in the buffer:

1) It *cuts* the buffer at i, splitting the buffer into two segments: (1) from the start to the end of the located tag, and (2) the bytes after that tag.
2) It returns the first segment as a chunk to add to the output, which the parser outputs or processes for the current state.
3) The second segment becomes the updated buffer, ensuring the XML node boundary is never split across segments.

If the parser fails to find any relevant tag or end of file, it returns no new output chunk, remains in the same state and leaves the buffer unmodified. This ensures that subsequent incoming data can be appended, eventually providing enough content to match the target boundary.

c) In column definition node: In IN_COLS, the parser searches for </cols> or any <col .../> elements. On detecting </cols>, it cuts the buffer right before the closing tag and transitions to BEFORE_SHEET_DATA. If it does not find the closing tag but locates complete <col .../> elements, those are consumed and removed from the buffer as a clean cut. If no match is found, the buffer remains unchanged, waiting for more data to complete the matching.

Algorithm 3: PreProcessSD — streaming prefilters for <sheetData> chunks

Data: $p_{\text{hid}} \leftarrow$ compile(
`'(<row\b [^/>]*?\bhidden ="1"[^/>]++' +`
`'(?:/>|>(?:(?!(?:</row>|<v>)).)++</row>))' +`
`'(?:<row\b [^/>]*?\bhidden ="1"[^/>]++' +`
`'(?:/>|>(?:(?!(?:</row>|<v>)).)++</row>))+')`

Function PREPROCESSSD*(b, $\mathcal{P}$)*:

- **if** has(b, *hidden="1"*) **then**
  - $b \leftarrow \text{sub}(p_{\text{hid}}, \text{"\1"}, b)$
- **if** has($\mathcal{P}.p_{ws}$) $\wedge$ has($\mathcal{P}.p_{wc}$) **then**
  - **if** match($\mathcal{P}.p_{ws}, b$) **then**
    - $b \leftarrow \text{sub}(\mathcal{P}.p_{\text{wc}}, \text{"\1/ > "}, b)$
- **if** has(b, *<c*) **then**
  - $b \leftarrow \text{sub}(\mathcal{P}.p_{\text{e}}, \text{""}, b)$
- **return** b

d) In sheet data node: While IN_SHEET_DATA, the parser similarly looks for </sheetData> or row boundaries <row ...>...</row> (or <row .../>). If </sheetData> is found, the parser cuts the buffer at that tag and transitions to AFTER_SHEET_DATA. If no boundary is present yet, the buffer remains unaltered to combine with future data chunks.

e) Ensuring "Clean Cuts" within the Streaming Buffer: Our parser reads raw XML data into a working buffer and ensures each XML node boundary is cleanly preserved, so no node is split across buffers. Each state-transition function adheres to this principle, only cutting when a relevant XML tag is fully identified.

f) Buffer Updates and State Progress: By constraining parsing decisions to these well-defined cuts, the parser never splits an XML node across multiple buffers. Each function (1) cuts on the matching tag boundary and updates the parser state, or (2) leaves the buffer intact if no complete boundary is found, thus allowing the next chunk to complete it. This way the parser can process the XML incrementally without needing to store the entire document in memory and it is ensured that the regular expressions dont't fail due to incomplete XML nodes and also only the regular expressions that are relevant to the current section of the document are applied, incresing the performance in comparison to applying all regular expressions on the whole document.

The sub-routine preProcessSheetData (Algorithm 3) performs three linear passes over chunks: (i) collapse explicit whitespace cells discovered via shared-strings indices, (ii) remove consecutive empty cells (either styleless or font-only), and (iii) opportunistically remove runs of hidden rows while retaining the first sentinel when needed to preserve row height/visibility semantics.

Two of these transformations depend on workbook-specific patterns derived from styles.xml and sharedStrings.xml as described in Section IV-B. Those are the removal of explicit whitespace cells and the removal of consecutive empty cells with font-only or no styling. Before these, we can also collapse consecutive hidden rows into a single sentinel row. In Excel's internal representation, rows can be marked as hidden using the attribute hidden="1". While such rows are visually collapsed in the spreadsheet, their presence in the XML can extend unnecessarily over thousands or even millions of entries. In practice, we observed worksheets where, starting at a certain point, all subsequent rows were marked as hidden. However, Excel renders all following rows as hidden as soon as a single row with hidden="1" is encountered, unless a later row explicitly overrides this setting. Therefore, from a rendering perspective, a long sequence of hidden rows is semantically equivalent to just the first hidden row being present. To exploit this, we use a regular expression to identify and collapse consecutive hidden rows into a single representative row. This technique avoids expensive per-row checks and requires no document-wide context, making it highly suitable for pre-processing. The resulting simplification significantly reduces the size of the document before it is fully loaded into memory. The regex used for this task is constructed and applied in Algorithm 3 first. Then the whitespace cell replacement and empty cell removal follow.

Algorithm 4: SplitSheetDataRanges — compute *cellsRange* and trailing *rowsRange*

Function SPLITSHEETDATARANGES($B_{\text{sheetData}}$, $\mathcal{P}$)**:**

$i_{\text{lastC}} \leftarrow \text{rfind}(B_{\text{sheetData}},\ \texttt{"<c "})$
if $i_{\text{lastC}} = -1$ **then**
  $i_{\text{cellsEnd}} \leftarrow -1$
else
  $j \leftarrow \text{find}(B_{\text{sheetData}},\ \texttt{"</row>"},\ i_{\text{lastC}})$
  if $j = -1$ **then**
    raise *Error: "no </row> after last <c"*
  $i_{\text{cellsEnd}} \leftarrow j + |\texttt{"</row>"}|$

$i_{\text{visAttr}} \leftarrow i_{\text{cellsEnd}}$
$m_{\text{last}} \leftarrow \bot$
foreach $m \in \text{finditer}(patterns.p_{\text{fbh}},\ B_{\text{sheetData}},\ start = i_{\text{cellsEnd}})$ **do**
  $m_{\text{last}} \leftarrow m$
if $m_{\text{last}} \neq \bot$ **then**
  $i_{\text{visAttr}} \leftarrow \max(i_{\text{visAttr}},\ \text{start}(m_{\text{last}}))$

$j_{\text{close}} \leftarrow \text{find}(B_{\text{sheetData}},\ \texttt{"</row>"},\ i_{\text{visAttr}})$
$j_{\text{self}} \leftarrow \text{find}(B_{\text{sheetData}},\ \texttt{"/>"},\ i_{\text{visAttr}})$
if $j_{\text{close}} \neq -1 \wedge j_{\text{self}} \neq -1$ **then**
  if $j_{\text{close}} < j_{\text{self}}$ **then**
    $i_{\text{rowsEnd}} \leftarrow j_{\text{close}} + |\texttt{"</row>"}|$
  else
    $i_{\text{rowsEnd}} \leftarrow j_{\text{self}} + |\texttt{"/>"}|$
else if $j_{\text{close}} = -1 \wedge j_{\text{self}} \neq -1$ **then**
  $i_{\text{rowsEnd}} \leftarrow j_{\text{self}} + |\texttt{"/>"}|$
else if $j_{\text{close}} \neq -1 \wedge j_{\text{self}} = -1$ **then**
  $i_{\text{rowsEnd}} \leftarrow j_{\text{close}} + |\texttt{"</row>"}|$
else
  raise *Error: "no </row> or /> after last anchor"*

if $i_{\text{cellsEnd}} = -1$ **then**
  cellsRange $\leftarrow \epsilon$
else
  cellsRange $\leftarrow B_{\text{sheetData}}[0 : i_{\text{cellsEnd}}]$
$i_{\text{rowsStart}} \leftarrow \max(0,\ i_{\text{cellsEnd}})$
if $i_{\text{rowsEnd}} = -1$ **then**
  rowsRange $\leftarrow \epsilon$
else
  rowsRange $\leftarrow B_{\text{sheetData}}[i_{\text{rowsStart}} : i_{\text{rowsEnd}}]$

return $\langle$*cellsRange*, *rowsRange*$\rangle$

D. Post-processing of `<sheetData>`

After streaming prefilters and structural range extraction, the post-processing phase transforms the `<sheetData>` payload into a compact but layout-equivalent representation. The phase comprises two linear passes that operate on disjoint byte slices returned by the splitter of Alg. 4: (i) a *rows+cells* sweep over the prefix slice that still contains cells (*cellsRange*), and (ii) a *trailing empty-rows* sweep over the suffix slice that no longer contains any cells (*rowsRange*). Both passes are guided by workbook-specific patterns (See IV-B) and update a small, per-sheet metrics state $\mathcal{M}$ used by later steps (merge-column normalization and diagnostics).

Range splitting: Alg. 4 identifies the last row that still contains a cell by scanning backwards for a `<c` token, then advancing to the end of that row. From that anchor it scans forward for the last *visibly relevant attribute*—either a fill/border-bearing style (`s="FB"`) or `hidden="1"`—by iterating

Algorithm 5: cleanRowsAndCellsInRange — apply p_{row} with cell cleanup

Data: $p_{\text{row}} \leftarrow$ compile(
`'<row\b ((?:[^>]*\br ="([^"]+?)")?' +`
`'(?:[^>]*\bhidden ="(1)")?' +`
`'(?:[^>]*\bs ="([^"]+?)")?' +`
`'(?:[^>]*\bht ="([^"]+?)")?' +`
`'[^/>]*?)(?:/>|>(.*?)</row>)'`)
$p_{\text{cell}} \leftarrow$ compile(
`'<c\b ((?:[^>]*\br ="([A-Z]+)(\d +)")?' +`
`'(?:[^>]*\bs ="(\d +?)")?[^/>]*?)/>'`)

Function CLEANROWSANDCELLSINRANGE(B_{row}, $\mathcal{P}$, $\mathcal{M}$, XFS)**:**
  return sub (B_{row}, p_{row}, REPLACEROW(·, $\mathcal{P}$, $\mathcal{M}$, XFS))

Function REPLACEROW(m, $\mathcal{P}$, $\mathcal{M}$, XFS)**:**
  $(rowAttr,\ r,\ hidden,\ sId,\ ht,\ content) \leftarrow \text{groups}(m)$
  $rowStyle \leftarrow (sId \neq \bot)\ ?\ \text{xfs}[sId]\ :\ \bot$
  $hasHeight \leftarrow (ht \neq \bot),\quad wasEmpty \leftarrow (content = \epsilon)$
  $\mathcal{M}.rowAttributes[r] \leftarrow (rowAttr,\ r,\ (hidden = 1),\ rowStyle,\ hasHeight,\ wasEmpty)$
  if $\neg wasEmpty$ **then**
    $content \leftarrow$ `CleanCells`($content$, $patterns$, $\mathcal{M}$, xfs)
  $isEmpty \leftarrow (content = \epsilon)$,
   $becameEmpty \leftarrow (\neg wasEmpty \wedge isEmpty)$
  if $hidden = 1$ **then**
    if *previousRowWasAlsoHidden*($\mathcal{M}$) **then**
      return ″
    else
      return `'<row' + clean(rowAttr) + '/>'`
  if $isEmpty$ **then**
    if $(rowStyle = \bot \vee \neg rowStyle.hasFillOrBorder) \wedge \neg hasHeight$ **then**
      return ″
    if $becameEmpty$ **then**
      $rowAttr \leftarrow$ ADDCUSTOMHEIGHTIFABSENT($rowAttr$)
    return `'<row' + clean(rowAttr) + '/>'`
  return `'<row' + rowAttr + '>' + content + '</row>'`

Function CLEANCELLS($rowBytes$, $\mathcal{P}$, $\mathcal{M}$, XFS)**:**
  if $rowBytes = \epsilon$ **then**
    return $rowBytes$
  return `sub` ($rowBytes$, p_{cell}, `ReplaceCell` (·, $\mathcal{M}$, XFS))

Function REPLACECELL(m, $\mathcal{M}$, XFS)**:**
  $(cellAttr,\ col,\ row,\ sId) \leftarrow \text{groups}(m)$
  $style \leftarrow (sId \neq \bot)\ ?\ \text{XFS}[sId]\ :\ \bot$
  $coord \leftarrow (col, row)$
  if CELLISHIDDEN($coord$, $\mathcal{M}$) **then**
    return ″
  if $style = \bot$ **then**
    return ″
  if $style.hasFillOrBorder$ **then**
    return `'<c' + cellAttr + '/>'`
  if $\neg style.hasApplyFillOrBorder$ **then**
    return ″
  if CELLOVERRIDESFILLORBORDER($style$, $coord$, $\mathcal{M}$) **then**
    return `'<c' + cellAttr + '/>'`
  else
    return ″

a precompiled pattern p_{fbh}. The earliest of the next `</row>` and `/>` delimiters after this anchor marks the end of the trailing visible-rows region. The function returns two slices: the *cells_range* from the beginning up to and including the last cell-bearing row, and the *rows_range* from there up to the last visibly relevant row. This design avoids unnecessary checks, as all content beyond the last relevant row is guaranteed to be empty and can therefore be removed without further inspection, whereas the intermediate range (between the last cell-bearing and the last relevant row) is examined only for rows that are hidden or have an explicitly defined height.

Row+cell sweep on the cells prefix: The first pass (Alg. 5) applies a row-level regex p_{row} to the *cellsRange*. Each row match is handed to *ReplaceRow*, which (i) records row attributes for later hidden/override checks, (ii) cleans the inner cells via *CleanCells* if present, and (iii) decides to delete the row, keep it as self-closing (empty but layout-relevant), or keep it with (cleaned) content. The cell policy is encapsulated in *ReplaceCell*: cells in hidden row/column, without style, or without visible effects are dropped; cells with fills/borders or with effective overrides are kept as self-closing. All actions are appended to $\mathcal{M}$ for downstream use.

Trailing empty-rows sweep: The second pass (Alg. 6) runs over the *rowsRange* using p_{rowE}. *ReplaceEmptyRow* preserves the first row of any hidden-run and any style-bearing empty row (to retain layout height/borders), while deleting layout-neutral empty rows. This pass never touches cells (by construction there are none in the suffix), which makes it both simpler and faster. Also p_{rowE} is slightly simpler and hence faster than p_{row} as it omits the cell content group.

Algorithm 6: CleanEmptyRowsFromAfterCellRange — apply p_{rowE}

```
Data: p_rowE ← compile(
'<row\b ((?:[^>]*\br ="([^"]+?)")?' +
'(?:[^>]*\bhidden ="(1)")?' +
'(?:[^>]*\bs ="([^"]+?)")?' +
'[^/>]*?)(?:/>|></row>)')

Function CleanEmptyRowsFromAfterCellRange(x, P, M, XFS):
    return sub (x, p_rowE, ReplaceEmptyRow (·, M, XFS))

Function ReplaceEmptyRow(m, M, XFS):
    (rowAttr, r, hidden, sId) ← groups(m)
    rowStyle ← (sId ≠ ⊥) ? xfs[sId] : ⊥
    M.rowAttributes[r] ← (rowAttr, r, (hidden =
      1), rowStyle, hasHeight = false, isEmpty = true)
    if hidden = 1 then
        if previousRowWasAlsoHidden(M) then
            return ''
        else
            return '<row' + clean(rowAttr) + '/>'
    if (rowStyle = ⊥ ∨ ¬hasFillOrBorder(rowStyle)) then
        return ''
    else
        return '<row' + clean(rowAttr) + '/>'
```

Metrics and column merging: The rows+cells pass records kept cell coordinates and row attributes in $\mathcal{M}$; combined with merge-cell bounds and visible column definitions, those are then used to merge extraneous `<col>` ranges (See IV-E).

a) Cell policy (summary): A cell is kept (as self-closing) iff it has a visible effect on layout: it is not in a hidden row or column, it carries a cell style, and it either applies a fill or border or explicitly overrides an inherited *apply* style from its row/column. Otherwise the cell is deleted.

E. Column Range Compaction

We compact `<cols>` by merging "extraneous" trailing definitions that exceed the rightmost visible column inferred from (i) kept cells, (ii) `<mergeCells>`, and (iii) last visible column definition (hidden or with fill/border). If the first extraneous range precedes others, we widen it to the last, replacing the suffix.

F. Workbook De-duplication of Defined Names

Some workbooks redundantly define both `Print_Area` and `_xlnm.Print_Area` (and analogously `Print_Titles`) for the same `localSheetId`. We keep one representative and delete duplicates.

G. Multi-process Execution

Our multi-process execution model relies on a process pool that distributes workbook and per-sheet cleaning tasks across multiple CPU cores. This level of concurrency is only feasible because `Excel Shrink` operates directly on the raw byte stream using regular expressions, rather than parsing the XML tree structure of each sheet.

If we instead used XML parsing—as in the earlier *Excel Shrink Analyzer* prototype—the memory footprint would already be saturated in single-process mode, with multi-gigabyte workbooks easily exhausting 8–16 GB of RAM on a typical consumer machine. Running 2, 4, or even 12 XML-based cleaning threads concurrently would therefore be impossible. By contrast, our byte-stream approach keeps memory consumption minimal and enables parallel execution even for extremely large workbooks.

Beyond memory efficiency, we further optimised scheduling. When a file with multiple sheets (e.g. 13) is being cleaned, up to 12 sheets can be processed in parallel. As soon as one process finishes its assigned sheet, it immediately fetches the next available task—either another sheet within the same workbook or, if all sheets are already processed, the next workbook in the queue. This dynamic scheduling ensures that no CPU core remains idle, except when the final workbook is being processed.

V. Evaluation

We tested our approach on 1,183 real Excel files from the German Federal Statistical Office (*Destatis*) [41], [42], covering year-based directories and containing numerous oversize spreadsheets. Excel Shrink Analyzer Script completed parsing and cleaning those files in 3 hours and 43 minutes while our multithreaded streambased approach using regular expressions took 20 minutes and 42 seconds. This represents

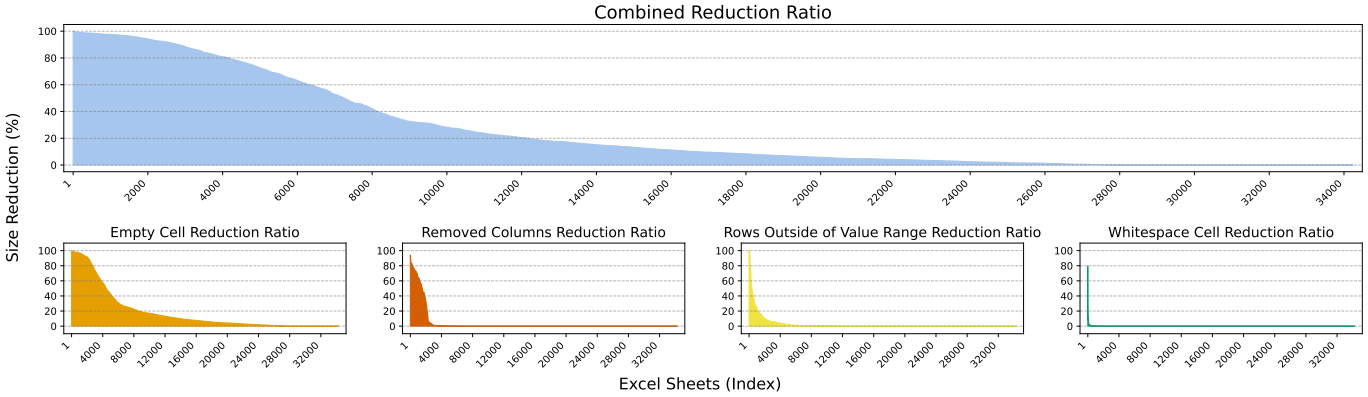


Fig. 1: Distribution of Size Reduction Percentages for Excel Sheets

a 10x speedup over the single-threaded DOM-Parser driven version. The multithreaded approach also significantly reduced memory usage, as each thread only required a small buffer to process its portion of the file. This improvement allowed us to process larger files that would have exceeded memory limits using the single-threaded approach.

Our primary goal was to quantify the extent of data reduction achieved by *Excel Shrink*, focusing on metrics such as overall size reduction, removal of empty cells, elimination of whitespace cells, deletion of rows outside the value range, and the removal of unnecessary columns. Figure 1 presents the distribution of reduction rates (in percentage) for individual worksheets across all these categories. The chart shows that some worksheets experienced reductions approaching 100% in file size. In particular, one worksheet exceeded a 99.99% reduction, 16 worksheets surpassed 99.9%, and 210 worksheets achieved reductions above 99%. Across all worksheets, the average reduction rate was 25%. Most of these gains were realized by removing rows that lay outside the range of actual data values.

To further assess the performance improvements enabled by *Excel Shrink*, we identified the two largest files in our *Destatis* dataset [43], [44]. Prior to processing, these files measured 85.2 MB and 36.3 MB in their compressed *Excel* (.xlsx) format and expanded to 911 MB and 741 MB when uncompressed. Figure 2 summarises the runtime, size, and memory consumption for the two representative *Destatis* workbooks before and after applying *Excel Shrink*. Each file was processed with six libraries—`openpyxl`, `pandas`, `Microsoft Excel`, `R openxlsx`, `R readxl+writexl`, and our own `Excel Shrink` pipeline—both in their original and cleaned versions.

a) Loading and Saving Times: The upper chart shows that `openpyxl` required by far the longest execution times, with more than 500 seconds for the largest workbook, whereas `pandas` and the R-based libraries performed substantially faster. After shrinking, the time to open and save the same files decreased for every library, often by one to two orders of magnitude. For example, `R readxl+writexl` improved from 191.7 s to 7.9 s in the second file, and `Excel Shrink` itself completed a full clean-and-save cycle in less than 10 seconds. The second shrink run ("Second Excel Shrink run") is shown only for completeness and confirms the idempotence of our cleaning approach—the processing time remains minimal because no further redundant content is detected.

b) File Size Reduction: The middle chart quantifies the reduction in file size. Both workbooks shrank substantially—from 85.2 MB and 35.4 MB (911 MB and 741 MB uncompressed) to 2.4 MB and 1.9 MB (20.8 MB and 10.7 MB uncompressed)—corresponding to 97.2% and 94.7% reductions (97.7% and 98.6% for the uncompressed sizes). Such compression not only reduces storage requirements but also decreases network transfer time when exchanging files.

c) Memory Consumption: The bottom chart shows the peak memory usage for each library before and after shrinking. Across all tools, the reduction in file complexity led to markedly lower memory requirements. `openpyxl` exhibited the highest demand—up to 8.5 GB when opening the uncleaned workbook—posing a serious challenge for consumer-grade computers with 8 GB of RAM. After shrinking, its peak memory dropped to about 620 MB, making the workbook manageable on typical systems. For R-based libraries such as `openxlsx` and `readxl + writexl`, the original peak consumption of roughly 2.5 GB decreased to below 700 MB. Even though `Excel Shrink` is implemented in Python, its memory profile was comparable to that of Microsoft Excel itself for the bloated workbook and became slightly lower than that of Excel for the cleaned workbook.

Overall, these results confirm that *Excel Shrink* not only minimises file sizes but also enables faster and more memory-efficient processing across different ecosystems, making large public spreadsheet datasets substantially more accessible for both analytical and production workloads.

A. Experimental Setup

To evaluate the behavior of *Excel* regarding how the files are interpreted and saved, as well as to test the behavior of the *Add-in* and the new *Check Performance* feature, we tested with two distinct versions of *Excel* from *Microsoft Office*: *Microsoft*

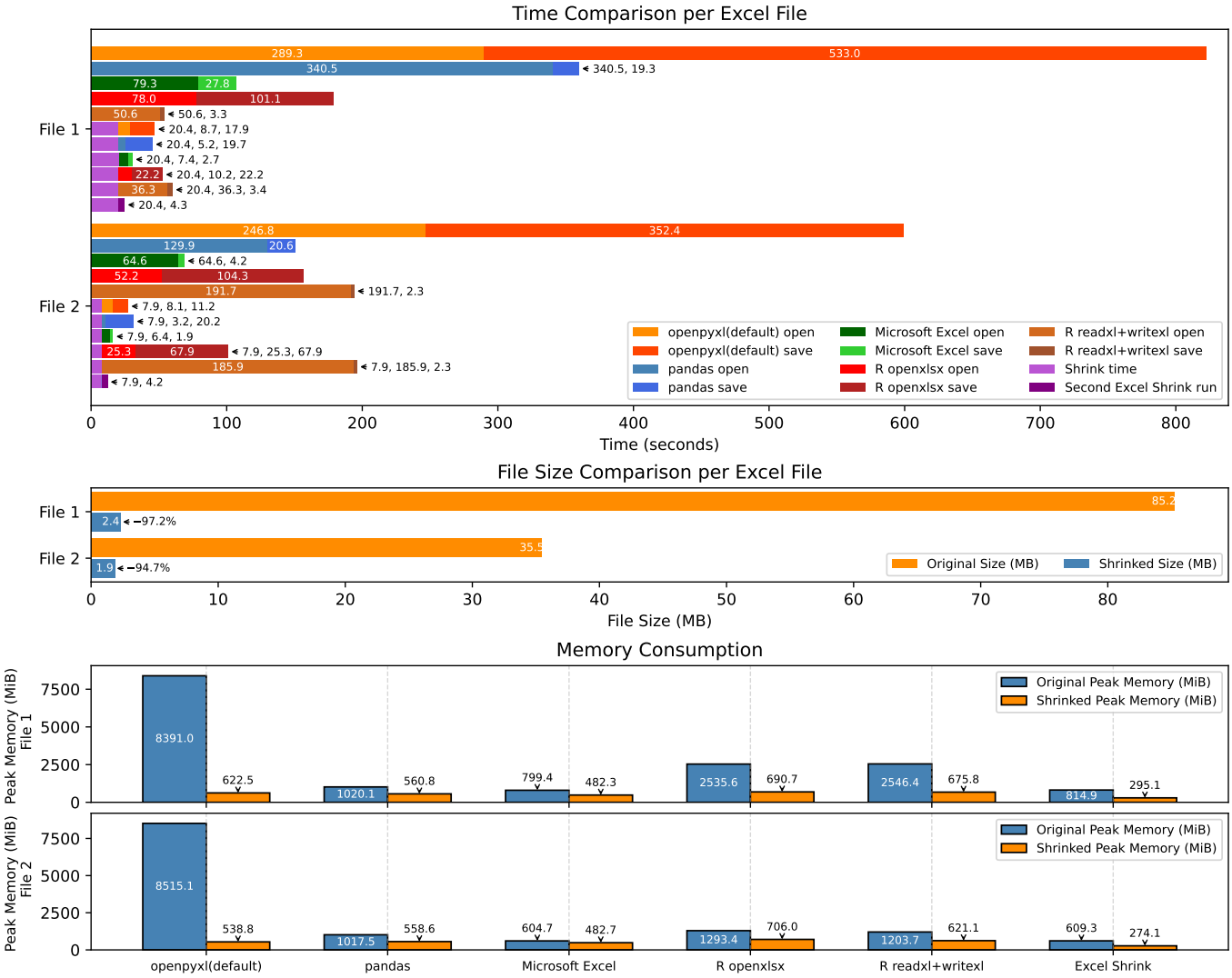


Fig. 2: Loading Times and Size Reduction for the 3 Largest *Destatis* Files Before and After applying *Excel Shrink*

Office LTSC Professional Plus 2021 and *Microsoft Excel for Microsoft 365 MSO (Version 2409 Build 16.0.18025.20030) 64-bit*. These versions were selected to evaluate compatibility and performance across different Excel environments. Our experiments were conducted on a laptop equipped with an AMD Ryzen 5 Pro 7535U processor (6 cores, 12 threads, 2.9 GHz base clock) and 32 GB of RAM.

B. Comparison with Other Tools for Reducing Bloat

Excel users lack a straightforward method to identify which cells in a worksheet contain only whitespace characters. To highlight these whitespaces, we have marked cells containing exclusively whitespaces in red within this worksheet, which includes 4,121 such instances (see Figure 3) [45]. Excel Inquire Add-in, Excel Check Performance and NXPowerLite Desktop did not remove Whitespaces, while Excel Shrink identified and removed them.

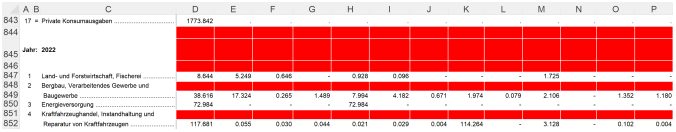


Fig. 3: Cells containing only whitespaces highlighted in red.

1) Excel Inquire Add-in: The *Excel Inquire* Add-in, available only in *Office Professional Plus* and *Microsoft 365 Apps for Enterprise*, can remove some redundant content [46].

Excel Inquire ignored whitespace cells but can delete empty cell, column, and row definitions. Yet, in certain cases, a significant number of unnecessary elements remain. For example, consider a worksheet serving as a cover sheet, where only the first 77 rows are used and the remaining rows (78 to 65,533(the old limit of row in Excelfiles before it was increased to 1,048,576)) are hidden. Despite being hidden, all these rows and cells are defined in the XML structure,

as shown in Listing 1.

```
  <row r="65533" spans="1:7" ht="0" hidden="1"
      customHeight="1">
    <c r="A65533" s="68" />
    <c r="B65533" s="68" />
    <c r="C65533" s="68" />
    <c r="D65533" s="68" />
    <c r="E65533" s="68" />
    <c r="F65533" s="68" />
    <c r="G65533" s="68" />
  </row>
</sheetData>
```

Listing 1: Row 65533 in sheetX.xml

When applying the *Excel Inquire* Add-in to such worksheets, these hidden rows and cells remain. A dialog box reports that because the default row height is `0`, the tool cannot clean the sheet, causing the operation to be skipped entirely. Excel Shrink successfully removed those cells.

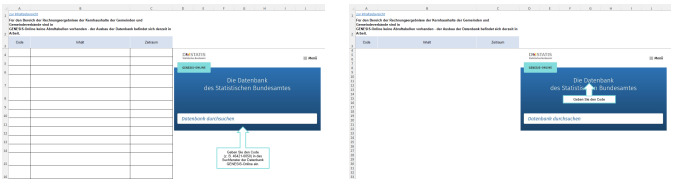
2) Excel Check Performance: The *Check Performance* feature, introduced in *Excel* for the Web and recent desktop versions, can achieve substantial file size reductions [47], [48]. However, it often removes cells and rows that influence the spreadsheet's layout and formatting, thereby altering its visual appearance. This is unacceptable in contexts where maintaining the original look and structure of the spreadsheet is essential.

Figure 4 illustrates how *Check Performance* modifies a cover sheet by removing empty rows. Such modifications become problematic when these empty rows serve a functional purpose, such as aligning the layout for printing. In the example shown, the cells responsible for providing space on the cover sheet (Figure 4a) are removed, as shown in Figure 4b [44]. Excel Shrink preserves this layout.

(a) Before Applying Check Performance (b) After Applying Check Performance

Fig. 4: Comparison of a Cover Sheet Before and After Applying Check Performance

As another example, consider a form-like structure composed of several empty cells arranged to form a table, anticipating future data entry [43]. *Check Performance* removes all borders outside the active data region, resulting in the scenario shown in Figure 5, where the intended table layout is lost. Because Excel Shrink marks all cells as important if they are visible to the user, it does not delete them.

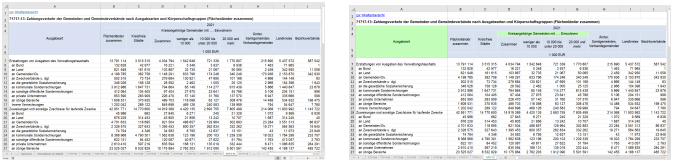


(a) Before Applying Check Performance (b) After Applying Check Performance

Fig. 5: Removal of Borders Defining an Empty Table Structure

3) LibreOffice Calc: Another commonly used tool capable of reading and rewriting Excel workbooks is *LibreOffice Calc*. When opening and re-saving spreadsheets, Calc internally parses the XML and reconstructs its own representation of the file, usually resulting in a smaller file size. However, LibreOffice does not fully interpret the `applyFill` and `applyBorder` overwrite flags, which can lead to visualisation errors when a workbook is re-saved.

Figure 6 shows such a case. In the original file (Figure 6a), header cells are shaded in light blue as intended. After saving the same workbook in LibreOffice Calc (Figure 6b), the shading changes to bright green because the `applyFill` flag is not respected. Excel Shrink preserves the original appearance.



(a) Before Saving with Libre Office Calc (b) After Saving with Libre Office Calc

Fig. 6: Visualisation Error Caused by Ignoring `applyFill` in LibreOffice Calc

VI. Conclusion

We have presented *Excel Shrink*, a tool that significantly reduces the file sizes of *Excel* sheets by eliminating unnecessary content. Our analysis identified common practices that lead to bloated *Excel* files and demonstrated how targeted removal of redundant data can achieve reductions of up to 99.99% in real-world Excel files without altering the spreadsheets' appearance while outperforming existing solutions in both file size reduction, preservation of formatting and performance.

Acknowledgements

I warmly thank Dr. agr. Norbert Röder and Dr. Johanna Fick for their invaluable guidance and advice on scientific writing, as well as Prof. Dr. Hardy Pundt and Prof. Dr.-Ing. Jörg Dörr for their mentorship during this work. I also wish to thank Linus Schmidt for his support in the scientific literature search and for the code experiments we conducted together. Finally, I extend my thanks to the Statistisches Bundesamt (DESTATIS) for enabling the publication of the insights presented here.

AI-Generated Content Acknowledgement

We used OpenAI's ChatGPT 5 solely for language editing and grammar enhancement. No AI-generated content was used in this paper; all scientific ideas, analyses, and conclusions are entirely the authors' own.

References

[1] M. Bendre, B. Sun, D. Zhang, X. Zhou, K. C.-C. Chang, and A. G. Parameswaran, "DataSpread: Unifying Databases and Spreadsheets," *PVLDB*, vol. 8, no. 12, pp. 2000–2003, 2015. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4756475/

[2] S. Rahman, M. Bendre, Y. Liu, S. Zhu, Z. Su, K. Karahalios, and A. G. Parameswaran, "NOAH: Interactive Spreadsheet Exploration with Dynamic Hierarchical Overviews," *PVLDB*, vol. 14, no. 6, pp. 970–983, 2021. [Online]. Available: https://doi.org/10.14778/3447689.3447701

[3] Microsoft, "Excel specifications and limits," n.d., accessed: 2025-02-28. [Online]. Available: https://support.microsoft.com/en-us/office/excel-specifications-and-limits-1672b34d-7043-467e-8e27-269d656771c3

[4] M. Budiu, P. Gopalan, L. Suresh, U. Wieder, H. Kruiger, and M. K. Aguilera, "Hillview: A Trillion-Cell Spreadsheet for Big Data," *PVLDB*, vol. 12, no. 11, pp. 1442–1457, 2019. [Online]. Available: https://doi.org/10.14778/3342263.3342279

[5] Microsoft Community, "Why did my excel file suddenly balloon in size?" Microsoft Answers forum post, 2011, accessed: 2025-10-15. [Online]. Available: https://answers.microsoft.com/en-us/msoffice/forum/all/why-did-my-excel-file-suddenly-balloon-in-size/4ed21955-dadb-481c-b976-31430df4a14c

[6] K. Mack, J. Lee, K. C.-C. Chang, K. Karahalios, and A. G. Parameswaran, "Characterizing Scalability Issues in Spreadsheet Software using Online Forums," in *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems (CHI EA '18)*. New York, NY, USA: ACM, 2018, pp. Paper CS04, 1–9. [Online]. Available: https://doi.org/10.1145/3170427.3174359

[7] M. Power, "Why is my Excel file so large and how to reduce Excel file size?" 2025, accessed: 2025-02-28. [Online]. Available: https://neuxpower.com/blog/why-is-excel-file-so-large-how-to-reduce-excel-file-size

[8] Microsoft, "You receive a 'Too many different cell formats' error message in Excel," 2024, accessed: 2025-03-01. [Online]. Available: https://learn.microsoft.com/en-us/office/troubleshoot/excel/too-many-different-cell-formats-in-excel

[9] akira_hirai, "Duplicate Styles warning," 2024, accessed: 2025-03-01. [Online]. Available: https://techcommunity.microsoft.com/discussions/excelgeneral/duplicate-styles-warning/4235042

[10] F. Henze, H. Gavriilidis, E. T. Zacharatou, and V. Markl, "Efficient Specialized Spreadsheet Parsing for Data Science," in *Proceedings of the 24th International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data (DOLAP)*, ser. CEUR Workshop Proceedings, vol. 3130. CEUR-WS.org, 2022, pp. 41–50. [Online]. Available: http://ceur-ws.org/Vol-3130/paper5.pdf

[11] sancozta, "Spout fails to read XLSX file with large sharedString.xml," 2020, accessed: 2025-03-01. [Online]. Available: https://github.com/box/spout/issues/731

[12] M. Champion, "Regular Expression Tips for People That Hate Regular Expressions," 2021, accessed: 2025-03-01. [Online]. Available: https://mattjameschampion.com/2021/02/06/regular-expression-tips-for-people-that-hate-regular-expressions/

[13] bobince, "RegEx match open tags except XHTML self-contained tags," 2009, accessed: 2025-03-01. [Online]. Available: https://stackoverflow.com/questions/1732348/regex-match-open-tags-except-xhtml-self-contained-tags/1758162#1758162

[14] Microsoft, "Structure of a SpreadsheetML document," 2025, accessed: 2025-03-01. [Online]. Available: https://learn.microsoft.com/en-us/office/open-xml/spreadsheet/structure-of-a-spreadsheetml-document?tabs=cs

[15] ——, "SharedStringTable Class (DocumentFormat.OpenXml.Spreadsheet)," 2025, accessed: 2025-03-01. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.spreadsheet.sharedstringtable?view=openxml-3.0.1

[16] FasterCapital, "SharedStrings: SharedStrings and Excel Open XML: A Guide to Optimized Text Management," 2024, accessed: 2025-03-01. [Online]. Available: https://www.fastercapital.com/content/SharedStrings--SharedStrings-and-Excel-Open-XML--A-Guide-to-Optimized-Text-Management.html

[17] Microsoft, "CellFormat Class (DocumentFormat.OpenXml.Spreadsheet)," 2025, accessed: 2025-03-01. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.spreadsheet.cellformat?view=openxml-3.0.1

[18] chrtak, "Unable to determine why my Excel Sheet is bloated," 2024, accessed: 2025-03-01. [Online]. Available: https://stackoverflow.com/questions/79280572/unable-to-determine-why-my-excel-sheet-is-bloated

[19] s_a, "How can I find out which parts of an Excel workbook are the biggest in size?" 2014, accessed: 2025-10-26. [Online]. Available: https://superuser.com/questions/739577/how-can-i-find-out-which-parts-of-an-excel-workbook-are-the-biggest-in-size

[20] Emre, "Excel file suddenly 35 MB?" 2009, accessed: 2025-10-26. [Online]. Available: https://superuser.com/questions/63119/excel-file-suddenly-35mb

[21] Anonymous, "File size grew unexpectedly large," n.d., accessed: 2025-10-26. [Online]. Available: https://www.excelforum.com/excel-general/1349596/file-size-grew-unexpectedly-large.html

[22] Ecma International, "Office Open XML Formats," Ecma International, Tech. Rep., 2007, accessed: 2025-02-27. [Online]. Available: https://www.ecma-international.org/wp-content/uploads/Office-Open-XML-White-Paper.pdf

[23] Microsoft Support, "The file size of an Excel workbook may unexpectedly increase exponentially," n.d., accessed: 2025-02-27. [Online]. Available: https://support.microsoft.com/en-us/topic/the-file-size-of-an-excel-workbook-may-unexpectedly-increase-exponentially-b1e8e0ad-94a8-f97f-5adc-2e22798beac9

[24] ——, "Reduce the file size of your Excel spreadsheets," Microsoft Support Article, saving a workbook as an Excel Binary Workbook (.xlsb) instead of the default Excel Workbook (.xlsx) can significantly reduce the file size. [Online]. Available: https://support.microsoft.com/en-us/office/reduce-the-file-size-of-your-excel-spreadsheets-c4f69e3a-8eea-4e9d-8ded-0ac301192bf9

[25] Reddit User, "Bloating excel files," 2018. [Online]. Available: https://www.reddit.com/r/excel/comments/c89f8x/bloating_excel_files/

[26] U. on MrExcel, "File size increased & slow after formulas and conditional formatting," accessed: 2025-02-26. [Online]. Available: https://www.excelforum.com/excel-general/762063-file-size-increased-and-slow-after-formulas-and-conditional-formatting.html

[27] UpSlide, "How to reduce excel file size in 8 easy ways," 2024, accessed: 2025-02-26. [Online]. Available: https://upslide.net/blog/reduce-excel-file-size/

[28] The Bricks, "How to Reduce Excel Sheet Size," Feb. 2025, accessed: 2025-02-27. [Online]. Available: https://thebricks.com/resources/guide-how-to-reduce-excel-sheet-size

[29] J. Weir, "Fixing bloated file size and slow calculation in excel," 2012. [Online]. Available: https://chandoo.org/wp/big-trouble-in-little-spreadsheet/

[30] Neuxpower, "Nxpowerlite desktop - file compressor software," 2024, accessed on October 7, 2024. [Online]. Available: https://neuxpower.com/nxpowerlite-desktop/

[31] Sobolsoft, "Excel file size reduce software," accessed: 2025-02-26. [Online]. Available: https://www.sobolsoft.com/excelsize/

[32] WeCompress, "Online file compressor for excel files," accessed: 2025-02-26. [Online]. Available: https://www.youcompress.com/excel/

[33] K. for Excel Repair, "Kernel for excel repair," accessed: 2025-02-26. [Online]. Available: https://www.reddit.com/r/excel/comments/1awtq8d/how_to_decrease_excel_file_size_and_speed_up/

[34] P. O'Beirne, "Excel 2013 spreadsheet inquire," in *Proceedings of the EuSpRIG 2013 Conference "Spreadsheet Risk Management"*. European Spreadsheet Risks Interest Group, 2013, accessed: 2025-02-26. [Online]. Available: http://www.sysmod.com/xltest

[35] "Excel 2024: Check performance in excel," 2024, accessed: 2025-02-26. [Online]. Available: https://www.mrexcel.com/excel-tips/excel-2024-check-performance-in-excel/

[36] Microsoft Support, "Cleanup cells in your workbook," n.d., accessed: 2025-02-26. [Online]. Available: https://support.microsoft.com/en-us/office/cleanup-cells-in-your-workbook-edcc579f-b82f-495b-8d31-e786cd11717b

[37] S. Rahman, K. Mack, M. Bendre, R. Zhang, K. Karahalios, and A. G. Parameswaran, "Benchmarking Spreadsheet Systems," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. New York, NY, USA: ACM, 2020, pp. 1589–1599. [Online]. Available: https://doi.org/10.1145/3318464.3389782

[38] M. Bendre, T. Wattanawaroon, K. Mack, K. C.-C. Chang, and A. G. Parameswaran, "Anti-Freeze for Large and Complex Spreadsheets: Asynchronous Formula Computation," in *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data (SIGMOD '19)*. New York, NY, USA: ACM, 2019, pp. 1277–1294. [Online]. Available: https://doi.org/10.1145/3299869.3319876

[39] P. Schauberger and A. Walker, *openxlsx: Read, Write and Edit xlsx Files*, R Foundation for Statistical Computing, Vienna, Austria, 2021, r package version 4.2.5. [Online]. Available: https://CRAN.R-project.org/package=openxlsx

[40] M. Bendre, V. Venkataraman, X. Zhou, K. C.-C. Chang, and A. G. Parameswaran, "Towards a Holistic Integration of Spreadsheets with Databases: A Scalable Storage Engine for Presentational Data Management," in *Proceedings of the 34th IEEE International Conference on Data Engineering (ICDE)*. Paris, France: IEEE, 2018, pp. 113–124. [Online]. Available: https://doi.org/10.1109/ICDE.2018.00020

[41] "Original destatis xlsx files before shrinking with excel shrink," accessed: 2024-12-07. [Online]. Available: https://anonymous.4open.science/r/original_destatis_xlsx_files-5B7B/

[42] "Destatis xlsx files shrunk with excel shrink," accessed: 2024-12-07. [Online]. Available: https://anonymous.4open.science/r/shrinked_destatis_xlsx_files-4E27/

[43] Statistisches Bundesamt Destatis (German Federal Statistical Office), "Statistischer Bericht - Rechnungsergebnisse der Kernhaushalte der Gemeinden und Gemeindeverbände 2021 (Statistical Report - Financial Results of the Core Budgets of Municipalities and Municipal Associations 2021)," September 2023. [Online]. Available: https://www.destatis.de/DE/Themen/Staat/Oeffentliche-Finanzen/Ausgaben-Einnahmen/Publikationen/Downloads-Ausgaben-und-Einnahmen/statistischer-bericht-rechnungsergebnis-kernhaushalt-gemeinden-2140331217005.html

[44] ——, "Verkehrsunfälle - Zeitreihen 2021 (Traffic Accidents - Time Series 2021) (Letzte Ausgabe - berichtsweise eingestellt)," July 2022. [Online]. Available: https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Verkehrsunfaelle/Publikationen/Downloads-Verkehrsunfaelle/verkehrsunfaelle-zeitreihen-pdf-5462403.html

[45] ——, "Private Konsumausgaben und Verfügbares Einkommen - 1. Vierteljahr 2023 (Private Consumption Expenditures and Disposable Income - 1st Quarter 2023) (Letzte Ausgabe - berichtsweise eingestellt)," June 2023. [Online]. Available: https://www.destatis.de/DE/Themen/Wirtschaft/Volkswirtschaftliche-Gesamtrechnungen-Inlandsprodukt/Publikationen/Downloads-Inlandsprodukt/konsumausgaben-pdf-5811109.html

[46] P. O'Beirne, "Excel 2013 spreadsheet inquire," *arXiv preprint arXiv:1401.7586*, 2014.

[47] P. Shirolkar, "Check performance in excel for the web," Sep. 2022, microsoft 365 Insider Blog, Accessed: 2024-04-27. [Online]. Available: https://techcommunity.microsoft.com/t5/microsoft-365-insider-blog/check-performance-in-excel-for-the-web/ba-p/4216095

[48] ——, "Make your workbooks more performant with check performance in excel for windows," Apr. 2024, accessed: 2024-10-18. [Online]. Available: https://techcommunity.microsoft.com/t5/microsoft-365-insider-blog/make-your-workbooks-more-performant-with-check-performance-in/ba-p/4224276